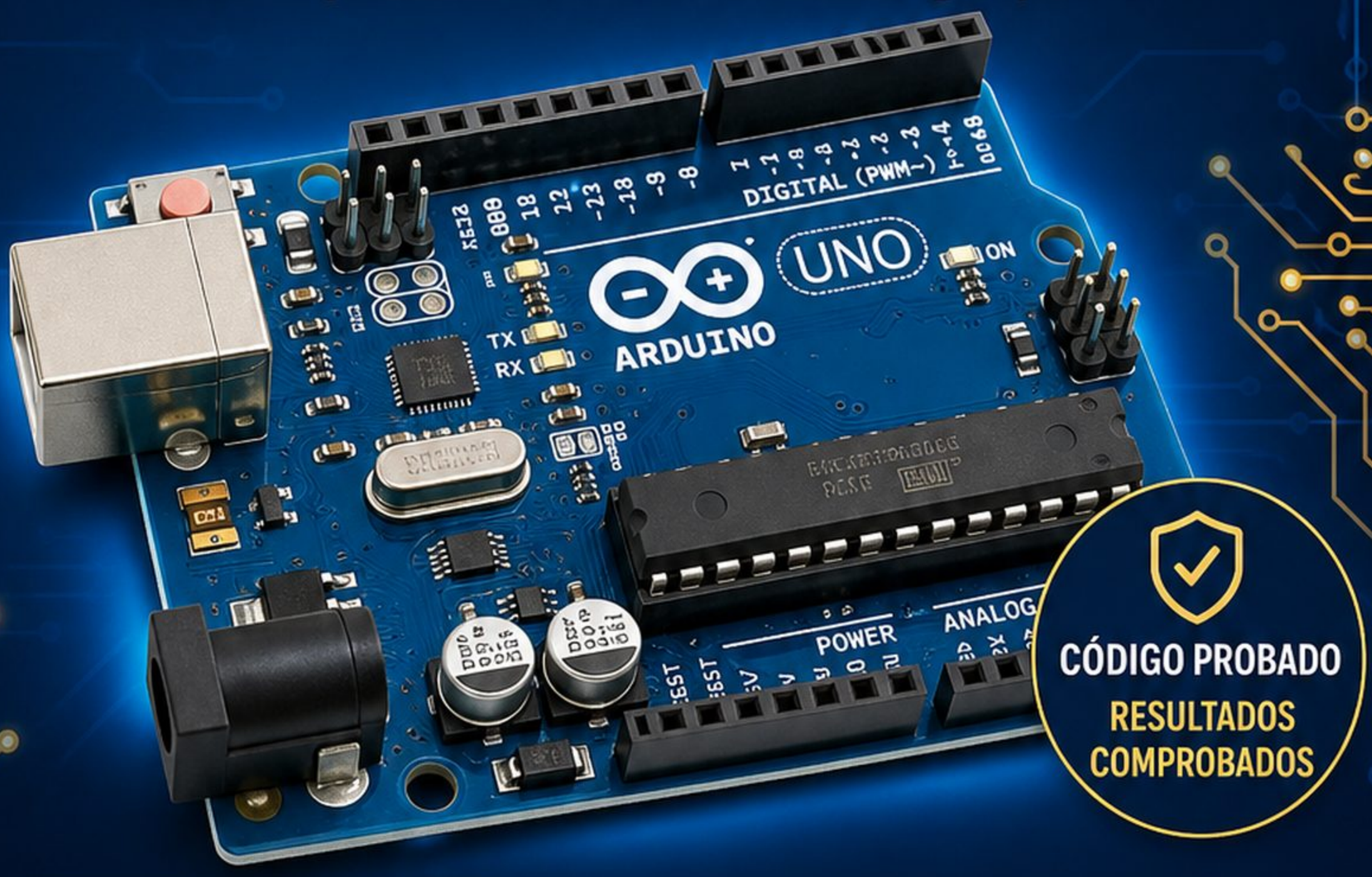
ARDUINO:
CÓDIGOS Y PLANTILLAS
LISTAS PARA USAR

Ahorra tiempo copiando y
personalizando **código probado**



CÓDIGO PROBADO
RESULTADOS COMPROBADOS

CÓDIGOS LISTOS PARA USAR

PLANTILLAS REUTILIZABLES

FÁCIL DE PERSONALIZAR

AHORRA TIEMPO Y ESFUERZO

Contenido



1. Introducción .....



2. Entradas y Salidas .....



3. Sensores .....



4. Actuadores .....



5. Comunicación .....



6. Pantallas .....



7. Control de Motores .....



8. Consejos y Trucos .....

Introducción

Las plantillas de código son una herramienta poderosa para ahorrar tiempo, evitar errores y acelerar el desarrollo de tus proyectos con Arduino. Estas plantillas proporcionan una base sólida y estructurada que puedes adaptar fácilmente a tus necesidades.

En este eBook encontrarás ejemplos listos para usar en una variedad de aplicaciones comunes. Cada plantilla está disenada para ser clara, eficiente y fácil de entender, incluso si estás comenzando en el mundo de la programación y la electrónica.

Nuestro objetivo es que pases menos tiempo escribiendo código desde cero y más tiempo creando soluciones increíbles.

¿Qué encontrarás?

Código probado

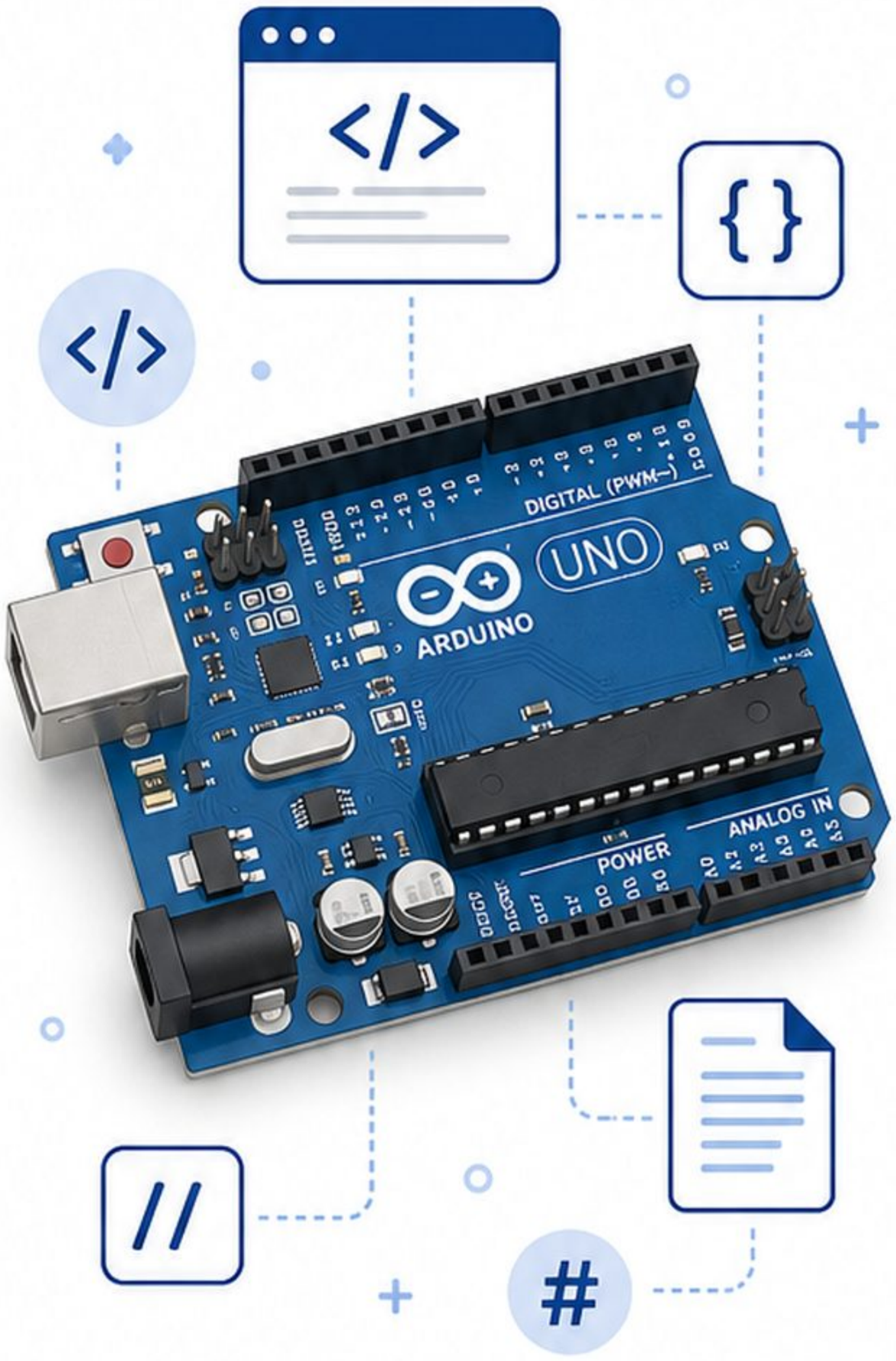
Ejemplos funcionales que han sido probados en escenarios reales.

Fácil de personalizar

Estructura flexible para que puedas adaptarlo a tus proyectos.

Comentarios claros

Explicaciones dentro del código para que entiendas cómo y por qué funciona.



Estructura Básica de un Sketch

Todo sketch de Arduino tiene dos funciones principales: **setup()** y **loop()**. Juntas forman la base de cualquier programa.

INICIO

setup()

Se ejecuta una sola vez al inicio.

setup()

Aquí configuras pines, comunicación serial, variables, etc.
Solo ocurre una vez al encender o reiniciar la placa.



loop()

Se ejecuta repetidamente para siempre.

loop()

Aquí colocas el código que quieres que se repita continuamente.
Es el corazón de tu programa.



EJEMPLO DE CÓDIGO </>

```
void setup() {
    pinMode(13, OUTPUT);
}

void loop() {
    digitalWrite(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```

setup()

Configura el pin 13 como salida.
Se ejecuta una sola vez.



loop()

Enciende el LED del pin 13 (HIGH), espera 1 segundo, lo apaga (LOW), espera 1 segundo y repite para siempre.



EN RESUMEN

setup() prepara tu sketch.

loop() hace que tu sketch haga cosas... ¡una y otra vez!

Entradas y Salidas Digitales

Las entradas y salidas digitales permiten que Arduino interactúe con el mundo exterior usando señales de dos estados: **HIGH (5V)** y **LOW (0V)**.

Función	Descripción	Sintaxis	Valores / Uso
pinMode()	Configura un pin como **ENTRADA** o **SALIDA**.	`pinMode(pin, modo);`	• pin: número de pin • modo: **INPUT** o **OUTPUT**
digitalWrite()	Escribe un valor digital (**HIGH** o **LOW**) en un pin configurado como **SALIDA**.	`digitalWrite(pin, valor);`	• pin: número de pin • valor: **HIGH** o **LOW**
digitalRead()	Lee el valor digital (**HIGH** o **LOW**) de un pin configurado como **ENTRADA**.	`digitalRead(pin);`	• pin: número de pin • Retorna **HIGH** o **LOW**

EJEMPLOS PRÁCTICOS

Ejemplo 1: Encender y apagar un LED

```
const int LED = 13;

void setup() {
  pinMode(LED, OUTPUT);    // Pin 13 como salida
}

void loop() {
  digitalWrite(LED, HIGH); // Enciende el LED
  delay(1000);             // Espera 1 segundo
  digitalWrite(LED, LOW);  // Apaga el LED
  delay(1000);             // Espera 1 segundo
}
```

Ejemplo 2: Leer un botón (pulsador)

```
const int BOTON = 2;

void setup() {
  pinMode(BOTON, INPUT_PULLUP); // Botón a GND
  pinMode(13, OUTPUT);          // LED en pin 13
}

void loop() {
  int estado = digitalRead(BOTON);
  if (estado == LOW) {          // Botón presionado
    digitalWrite(13, HIGH);     // LED encendido
  } else {
    digitalWrite(13, LOW);      // LED apagado
  }
}
```

COMPONENTES



Arduino UNO
Placa de desarrollo



LED
Diodo emisor de luz

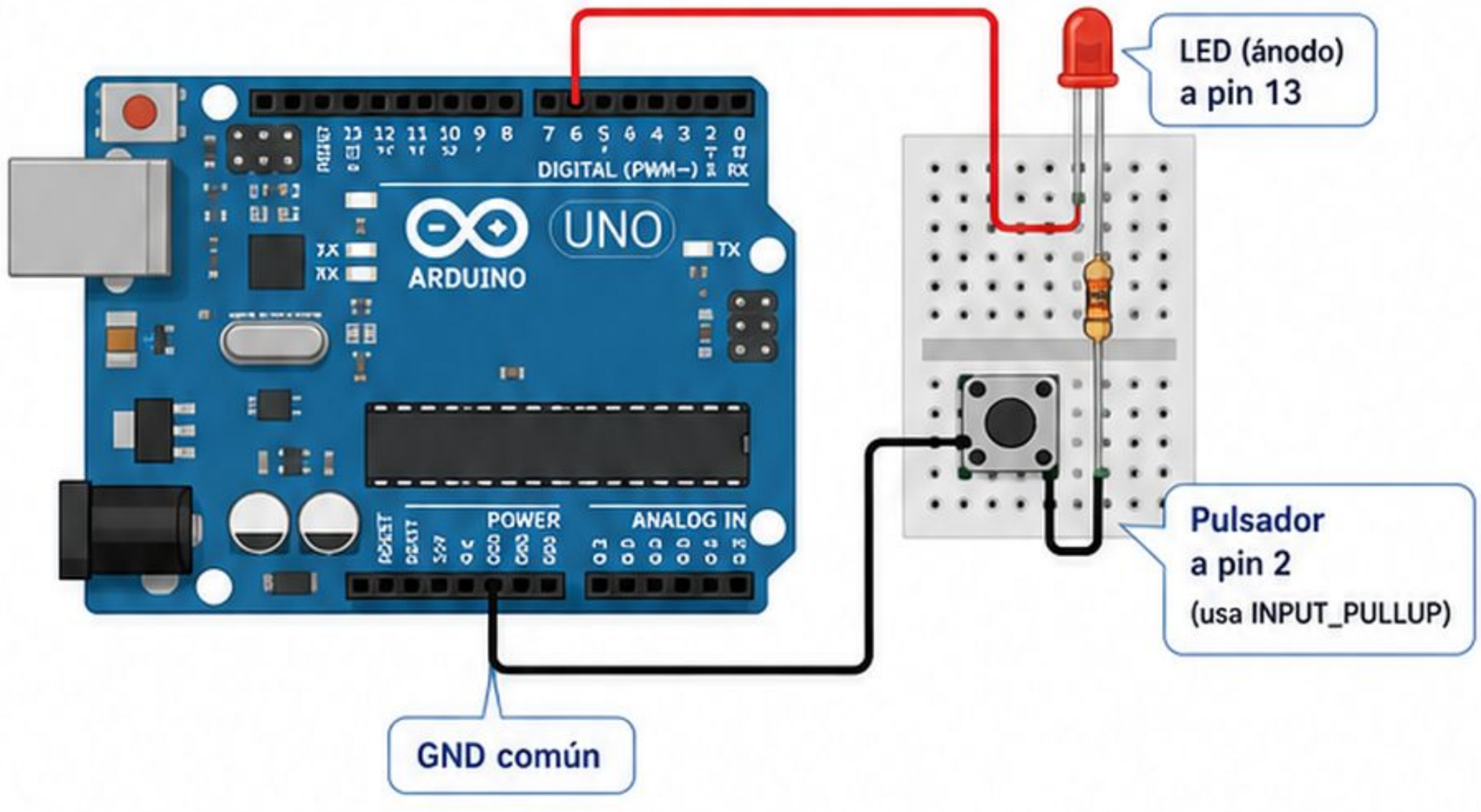


Pulsador
Botón momentáneo



Resistencia 220Ω
Limitadora de corriente

DIAGRAMA DE CONEXIÓN



CONSEJO PRÁCTICO: Cuando uses `INPUT_PULLUP`, conecta el botón entre el pin y GND. El pin leerá **HIGH** cuando no se presiona y **LOW** cuando se presiona.

Entradas Analógicas

Arduino puede leer señales analógicas a través de sus pines analógicos (A0 a A5). Estas señales pueden variar entre 0V y 5V, como las que entregan sensores o potenciómetros.



Sensores analógicos

¿Qué hace `analogRead()`?

La función `analogRead(pin)` lee el voltaje presente en un pin analógico y lo convierte en un valor entero `(int)` entre `0` y `1023`.

Ejemplo: Leer un potenciómetro

Para leer el valor de un potenciómetro conectado al pin A0:

```
int valor = analogRead(A0);
```

La variable `valor` almacenará un número entre `0` y `1023`.

Importante

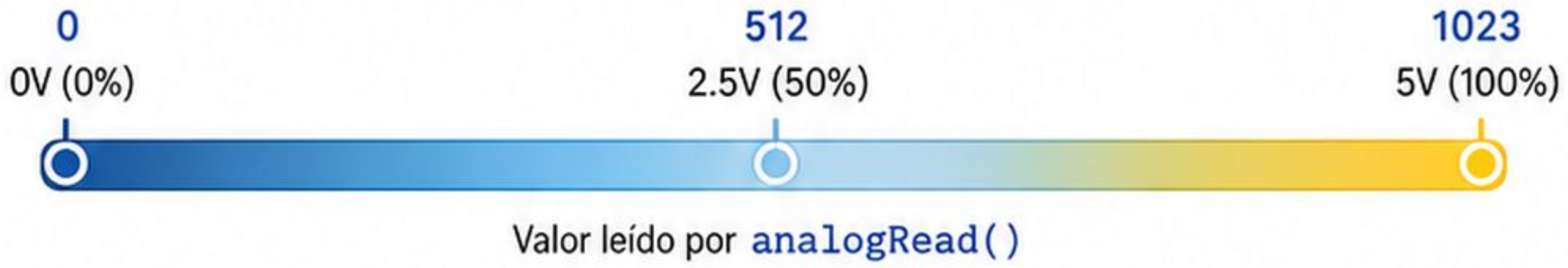
`0` representa 0V (0%)

`1023` representa 5V (100%)

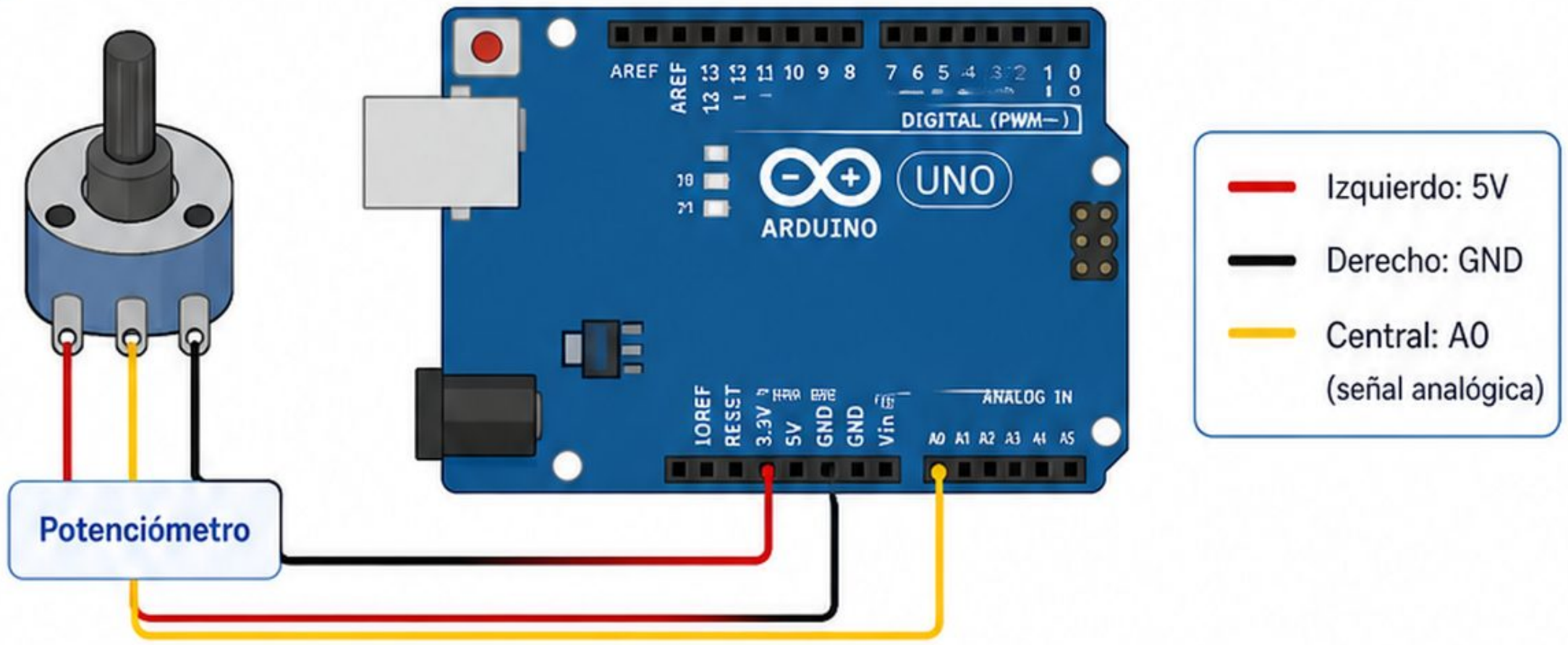
Puedes convertir el valor leído a voltaje con la fórmula:

Voltaje = (valor * 5.0) / 1023.0

Rango de valores



Conexión: Potenciómetro a Arduino



Consejo

Puedes usar estos valores para controlar luces, leer sensores, ajustar configuraciones o cualquier aplicación que requiera mediciones variables.

Salidas PWM

PWM

PWM (Modulación por Ancho de Pulso) permite simular una salida analógica variando el ciclo de trabajo de una señal digital de alta frecuencia. Esto es ideal para controlar el brillo de LEDs, la velocidad de motores, entre otros dispositivos.

¿Qué es `analogWrite()`?

La función `analogWrite(pin, valor)` genera una señal PWM en el pin especificado. El valor debe estar entre 0 y 255:

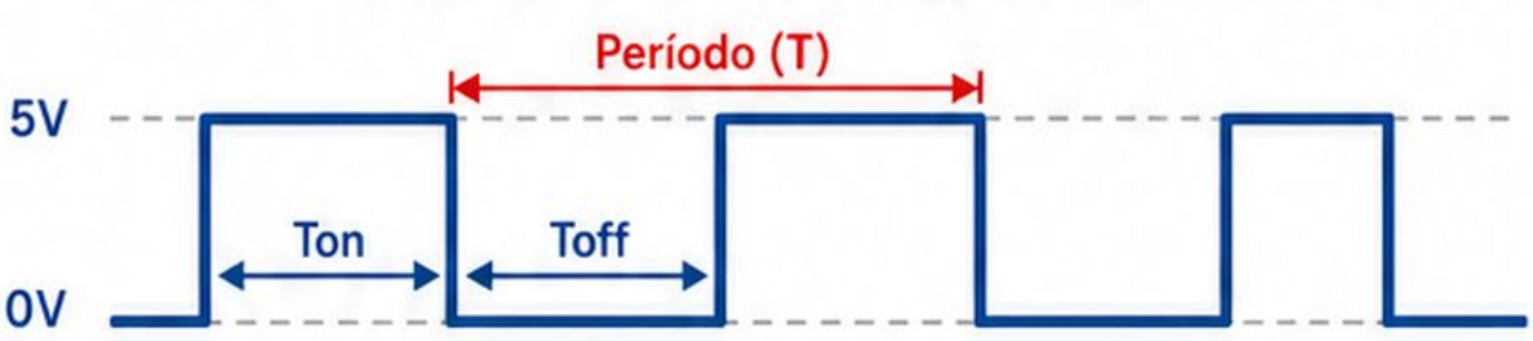
- **0** = 0% ciclo de trabajo (LED apagado)
- **128** = 50% ciclo de trabajo (brillo medio)
- **255** = 100% ciclo de trabajo (LED encendido)

Ejemplo: Controlar el brillo de un LED

```
// Establece el brillo del LED al 50%
analogWrite(9, 128);  // Valor entre 0 y 255
```

¿Cómo funciona?

PWM cambia rápidamente entre HIGH (5V) y LOW (0V).
El brillo depende del ciclo de trabajo (tiempo en HIGH).



Ciclo de trabajo (%)

$$D = \frac{T_{on}}{T} \times 100$$

Donde:
$T = T_{on} + T_{off}$

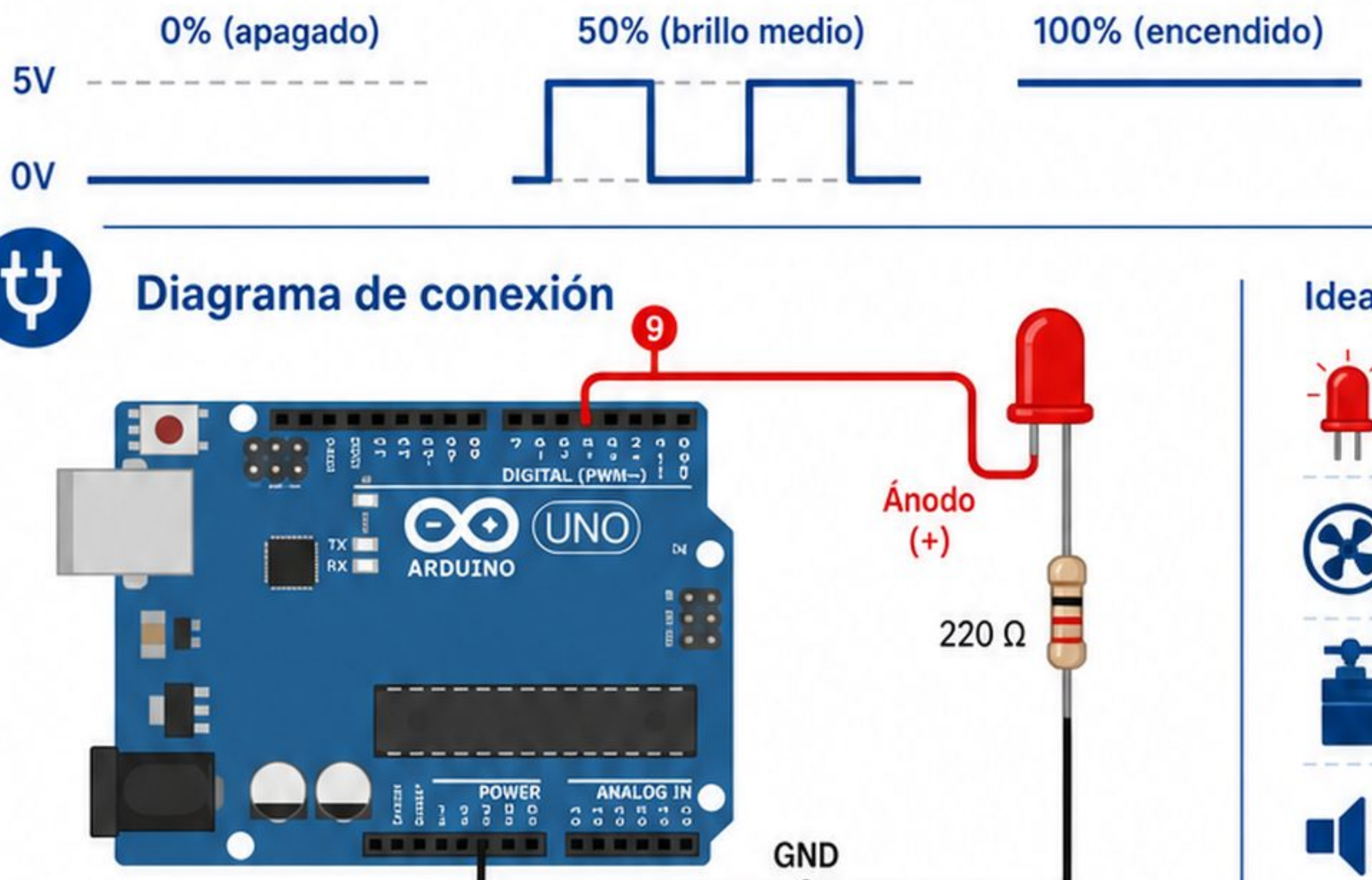


Diagrama de conexión

Ideal para controlar:

- Brillo de LEDs
- Velocidad de motores
- Posición de servos
- Tono y volumen

Consejo: Los pines PWM (~) en Arduino UNO son: 3, 5, 6, 9, 10 y 11. Usa siempre una resistencia en serie con el LED para protegerlo.

Sensores:
Temperatura y Humedad

El sensor **DHT11** permite medir la temperatura (°C) y la humedad relativa (%) del ambiente.

CÓDIGO DE EJEMPLO

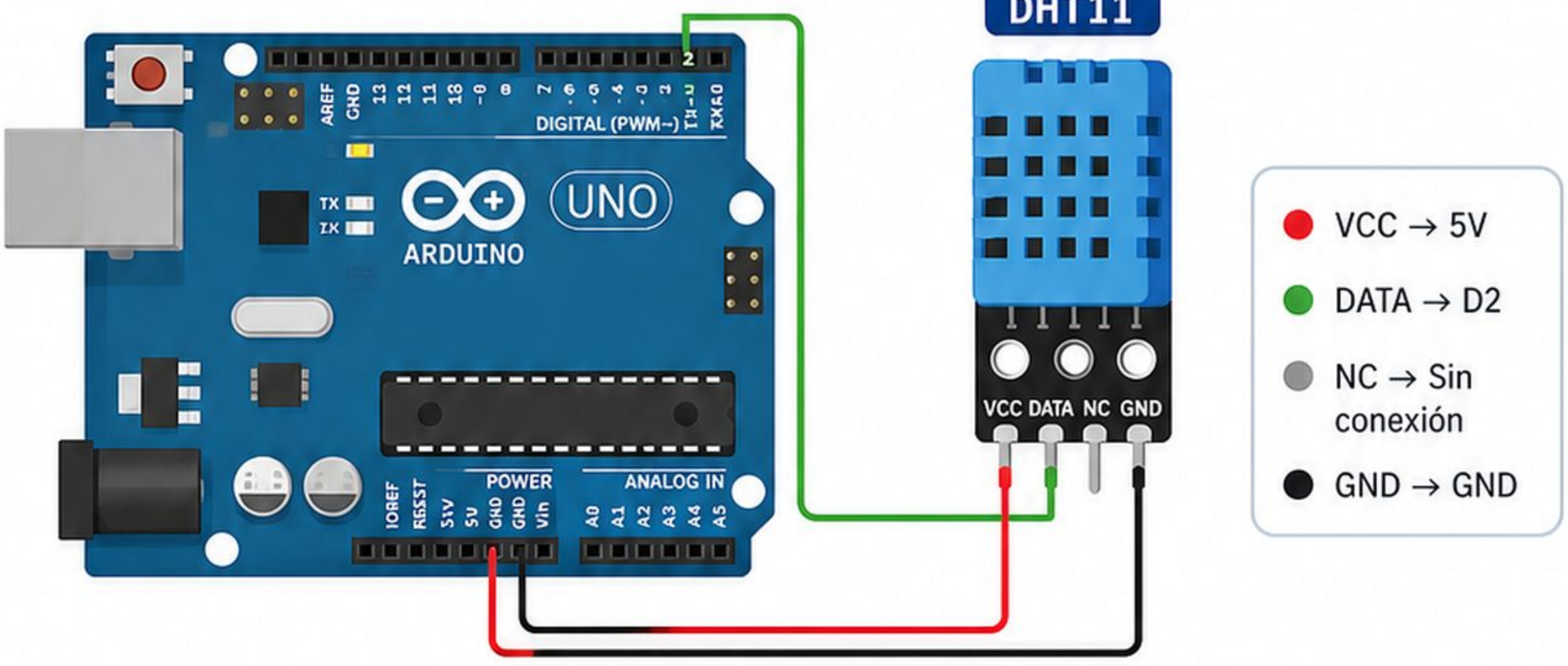
```
#include <DHT.h>                    // Incluye la librería DHT

DHT dht(2, DHT11);                  // Pin de datos: 2
                                    // Tipo de sensor: DHT11

dht.readTemperature();              // Lee la temperatura (°C)

dht.readHumidity();                 // Lee la humedad (%)
```

DIAGRAMA DE CONEXIÓN



Consejo práctico

Espera 1 segundo entre lecturas para obtener valores más estables y precisos.

Sensores: Ultrasonido

¿Qué es?

El sensor ultrasónico mide la distancia a un objeto utilizando ondas de sonido de alta frecuencia. Emite un pulso ultrasónico y mide el tiempo que tarda en regresar el eco.

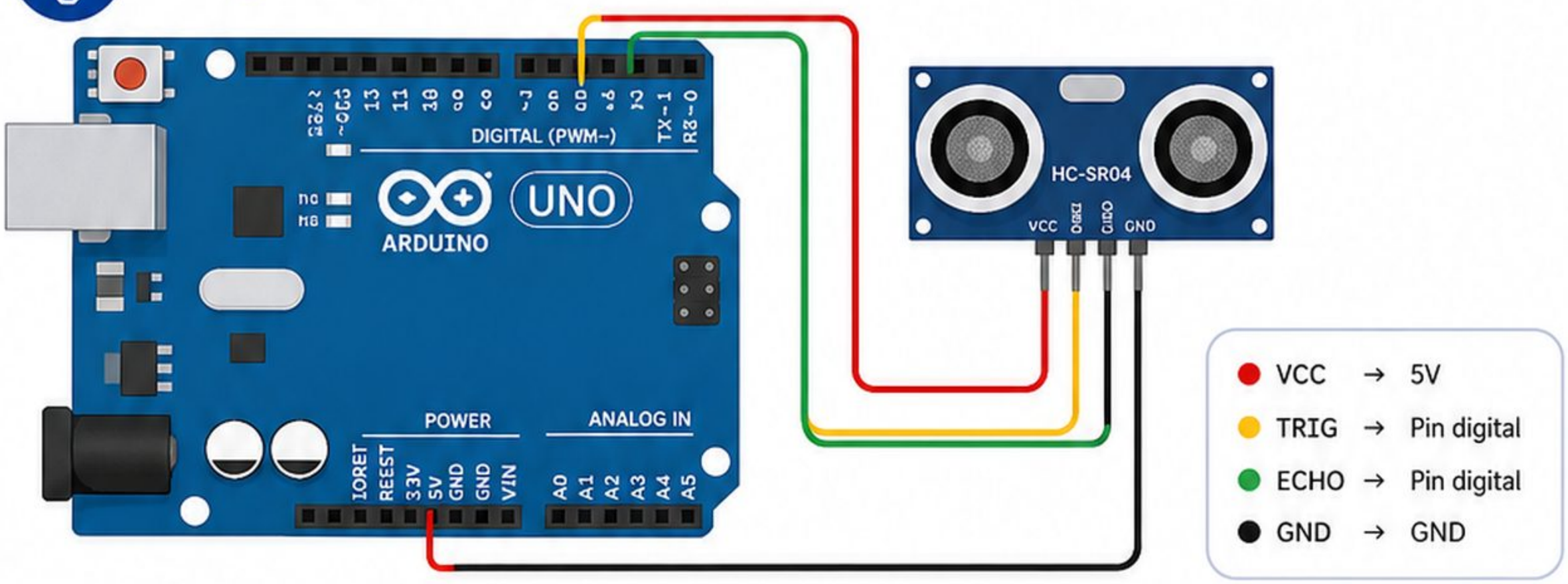


Código de ejemplo (Arduino)

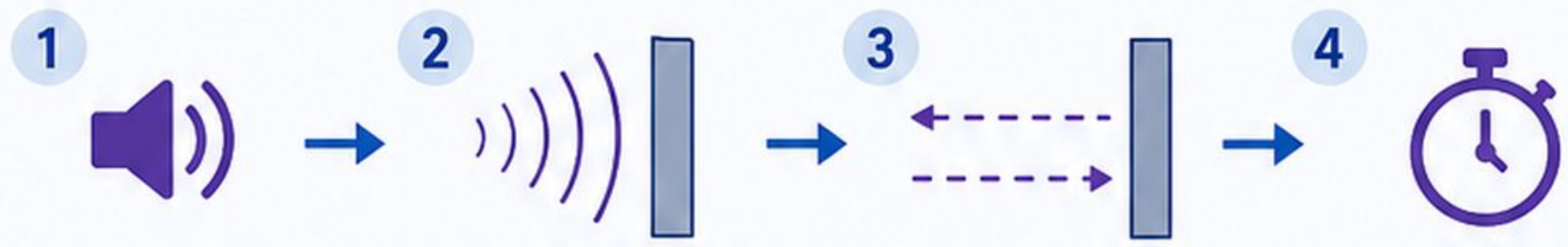
```
digitalWrite(trig, HIGH);
pulseIn(echo, HIGH);
distancia = duracion * 0.034 / 2;
```

- `trig`: pin de disparo
- `echo`: pin de eco
- `duracion`: tiempo en µs
- `distancia`: distancia en cm

Diagrama de conexión



¿Cómo funciona?



1. El sensor envía un pulso ultrasónico.
2. El pulso viaja por el aire y choca con un objeto.
3. El eco regresa al sensor y se detecta.
4. El sensor mide el tiempo del eco y calcula la distancia.

Fórmula: distancia (cm) = tiempo (µs) × 0.034 / 2
(0.034 cm/µs es la velocidad del sonido en el aire)

Actuadores: Servomotor



Un servomotor es un actuador que permite controlar la posición angular de su eje con precisión.

</> CÓDIGO DE EJEMPLO

```
1  #include <Servo.h>

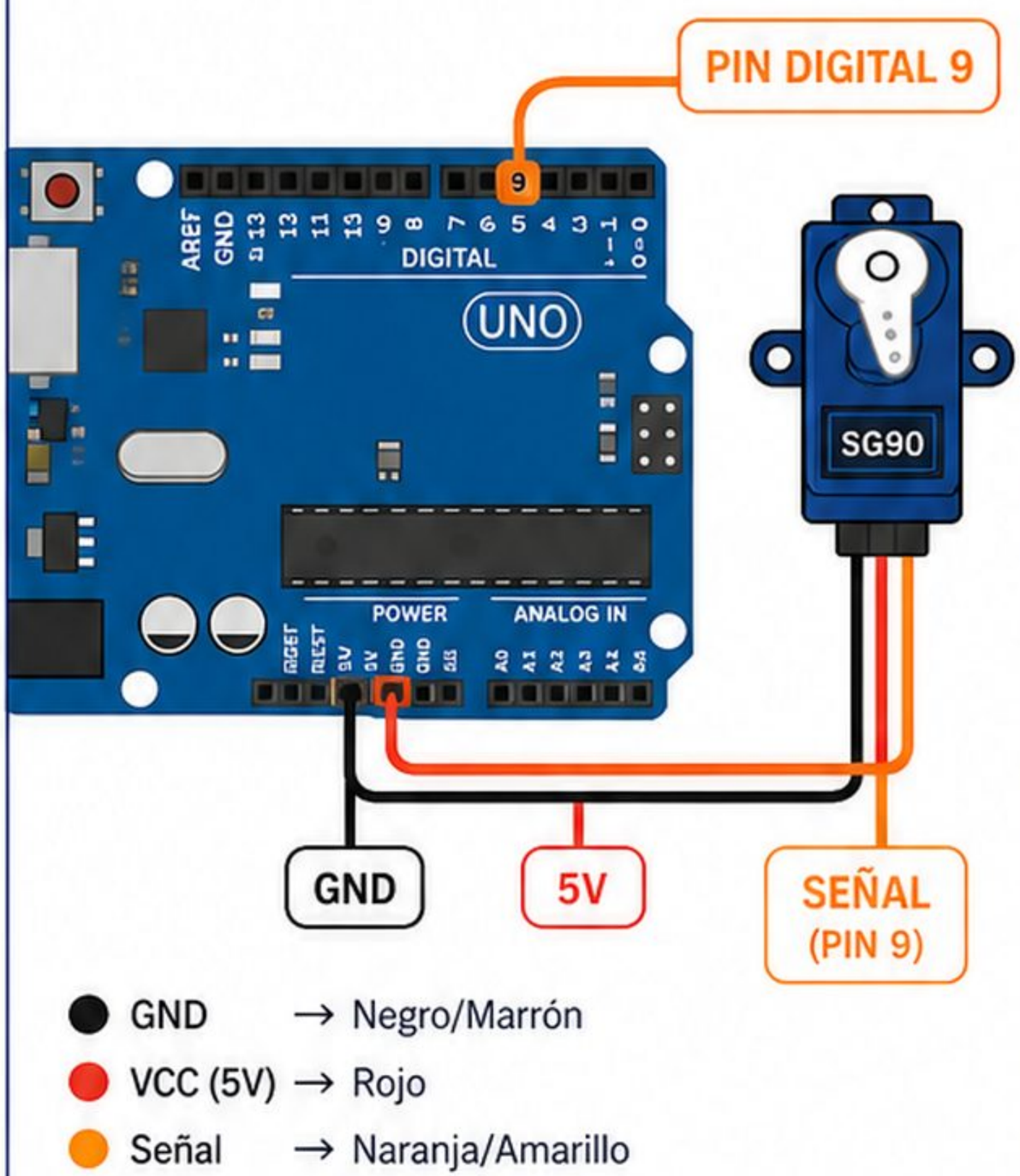
2  Servo myservo;

3  myservo.attach(9);

4  myservo.write(90);
```

1. Incluye la librería **Servo**.
2. Crea un objeto de tipo **Servo**.
3. Conecta el servomotor al **pin digital 9**.
4. Mueve el servo a **90 grados** (posición central).

DIAGRAMA DE CONEXIÓN



RANGO DE MOVIMIENTO

Un servomotor común puede moverse aproximadamente entre **0°** y **180°**.



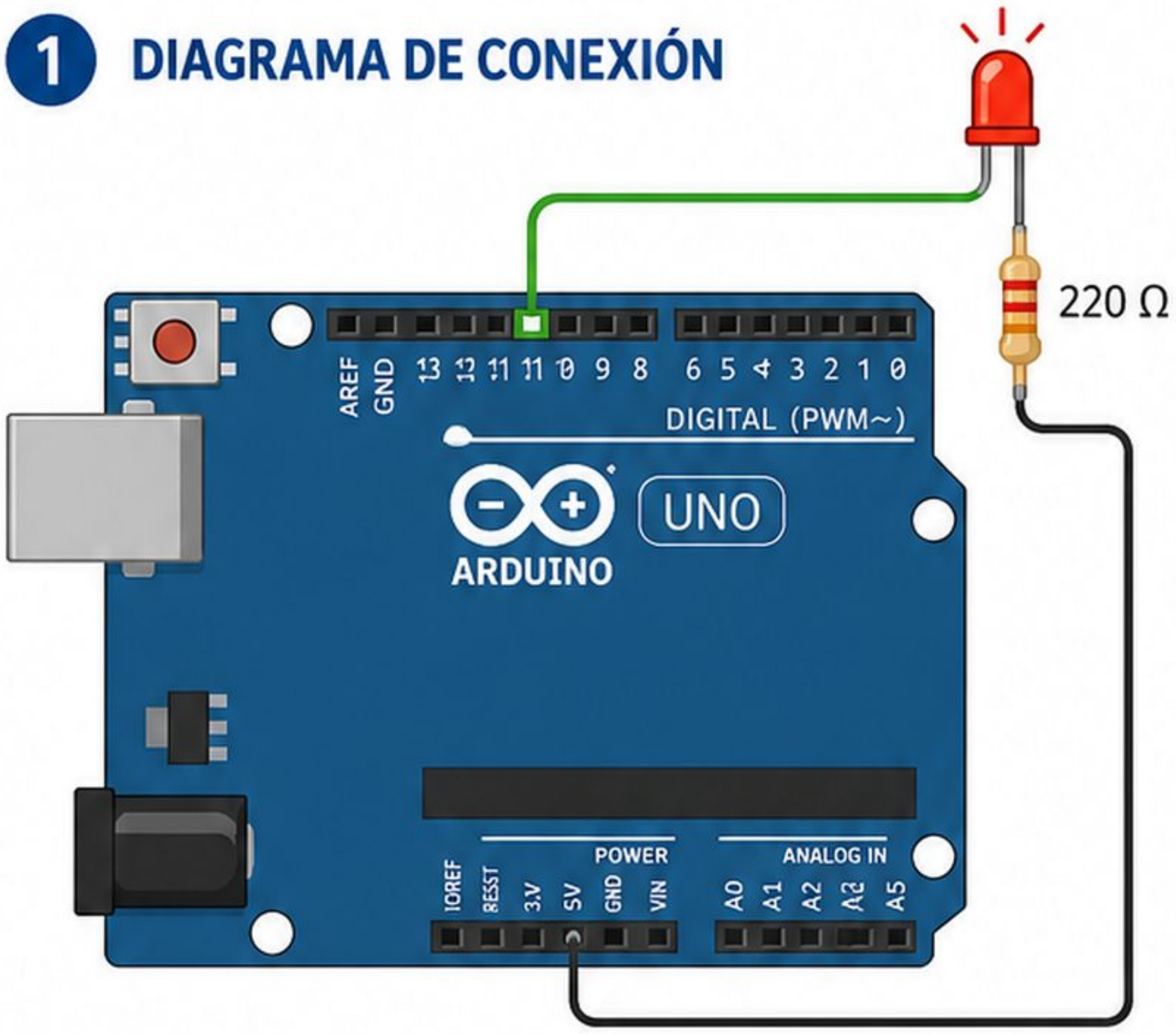
CONSEJO PRÁCTICO

Los valores de `myservo.write()` van de 0 a 180. Ajusta el valor para controlar la posición exacta del servomotor según tu proyecto.

Plantilla: Encender y Apagar un LED

Esta plantilla enciende un LED conectado al pin digital 13 durante 1 segundo y luego lo apaga durante 1 segundo, en un bucle infinito.

1 DIAGRAMA DE CONEXIÓN



CONEXIONES

- Pin digital 13 → Resistencia (220 Ω) → Ánodo (pata larga) del LED
- Cátodo (pata corta) del LED → GND

COMPONENTES

- 1 x LED
- 1 x Resistencia 220 Ω
- 1 x Arduino (Uno o compatible)
- Cables jumper

2 CÓDIGO DE EJEMPLO

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);    // Enciende el LED
  delay(1000);               // Espera 1 segundo
  digitalWrite(13, LOW);     // Apaga el LED
  delay(1000);               // Espera 1 segundo
}
```

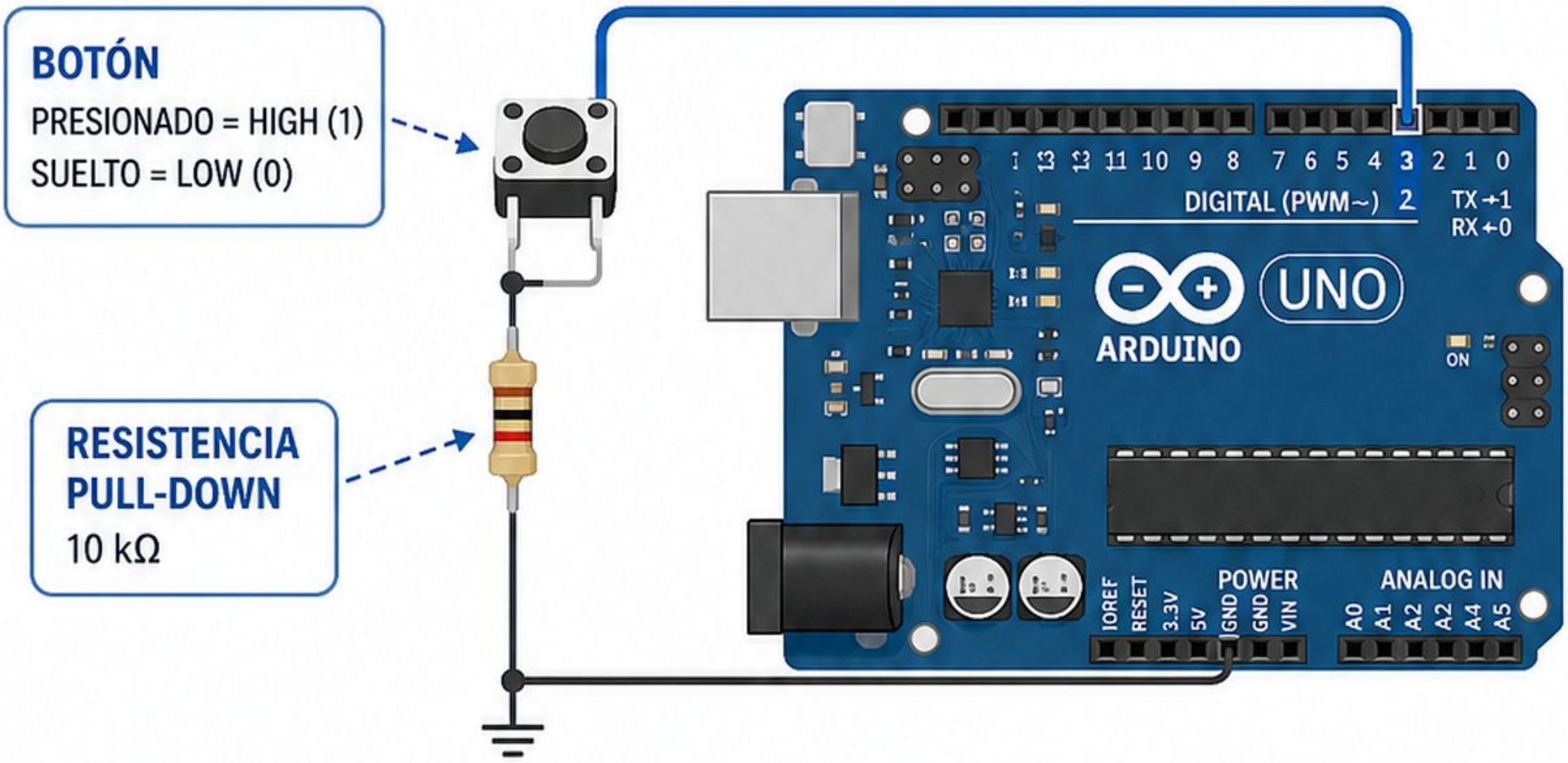
¿CÓMO FUNCIONA?

El pin 13 se configura como salida. El LED se enciende (HIGH) durante 1 segundo, luego se apaga (LOW) durante 1 segundo, y esto se repite indefinidamente.

Plantilla: Leer un Botón

Lee el estado de un botón conectado a un pin digital usando una resistencia **pull-down**.

DIAGRAMA DE CONEXIÓN

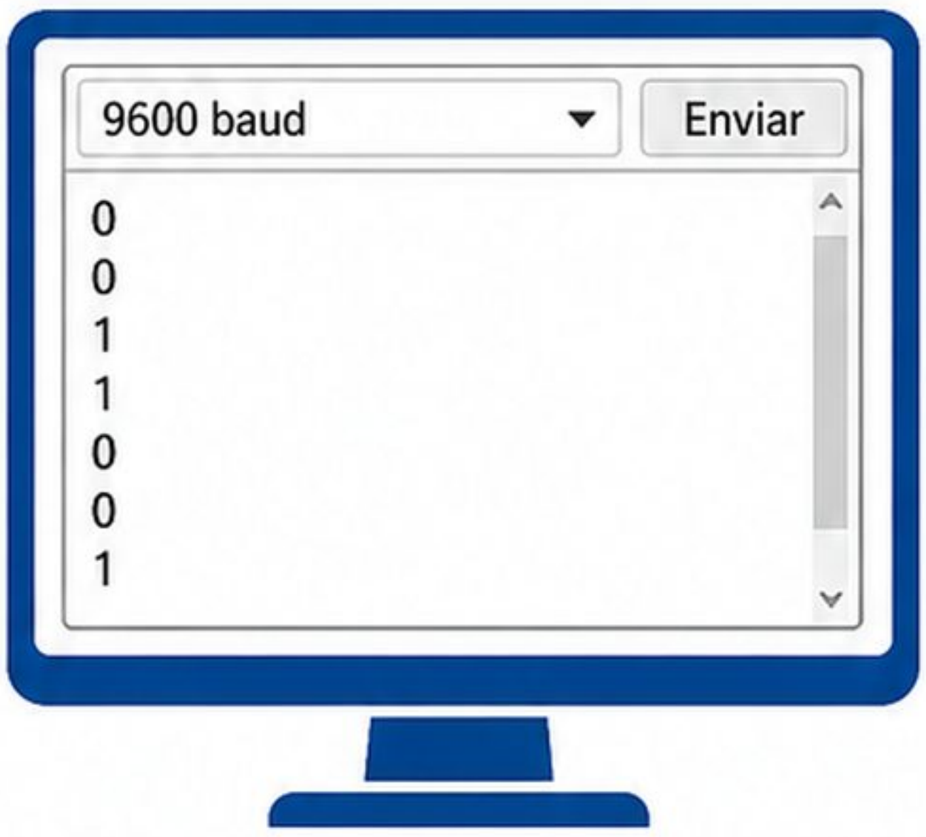


CÓDIGO DE EJEMPLO

```ino
void setup() {
  pinMode(2, INPUT);
  Serial.begin(9600);
}

void loop() {
  int estado = digitalRead(2);
  Serial.println(estado);
  delay(100);
}
```

MONITOR SERIE



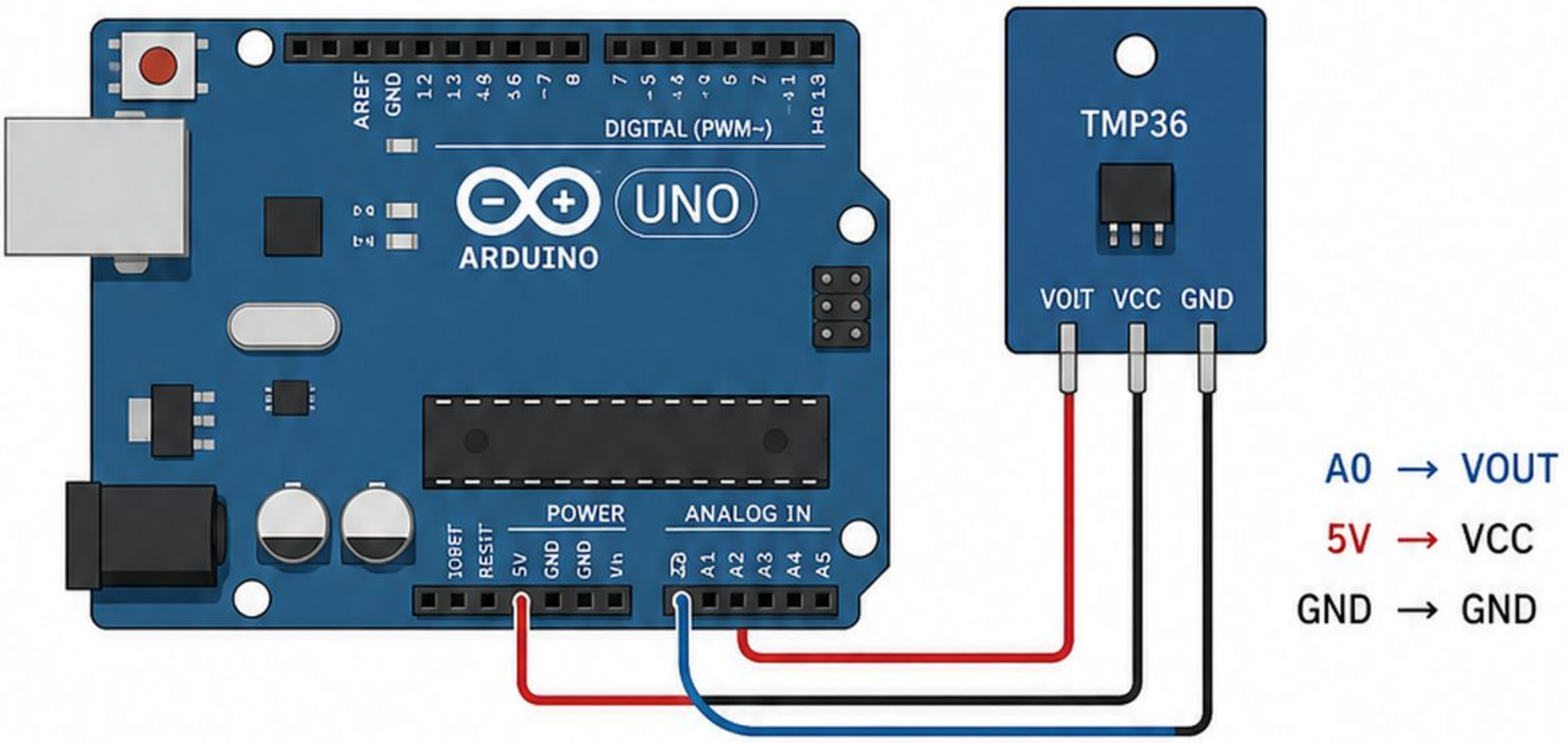
CÓMO FUNCIONA

La resistencia **pull-down** mantiene el pin en **LOW (0)** cuando el botón está suelto. Al presionarlo, el pin se conecta a **5V** y se lee **HIGH (1)**.

Plantilla: Medir Temperatura

Lee la temperatura desde un sensor analógico y la muestra por el Monitor Serie.

DIAGRAMA DE CONEXIÓN



CÓDIGO DE EJEMPLO

```ino
int pin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(pin);
  float temp = (valor * 5.0 / 1023.0) * 100;
  Serial.println(temp);
  delay(1000);
}
```

El TMP36 entrega 10 mV por °C. Con 5V de referencia, el rango 0–100 °C se mapea a 0–1023 en la lectura analógica.

ARDUINO

Plantilla: Comunicación Serial con PC

Esta plantilla permite recibir datos desde el Monitor Serial del IDE de Arduino y devolverlos a la PC.

1. CONEXIÓN



Puerto USB

Arduino se comunica con la PC a través del puerto USB. Asegúrate de tener instalado el controlador (driver) correspondiente.

Monitor Serial

Abre el Monitor Serial en el IDE de Arduino y configura la misma velocidad (baud rate).

2. CÓDIGO DE EJEMPLO

```ino
void setup() {
  Serial.begin(9600);  // Inicia la comunicación serial a 9600 baudios
}

void loop() {
  if (Serial.available()) {
    char c = Serial.read();     // Lee un carácter recibido
    Serial.print("Recibido: "); // Envía texto a la PC
    Serial.println(c);          // Envía el carácter y salto de línea
  }
}
```

3. CÓMO PROBARLO

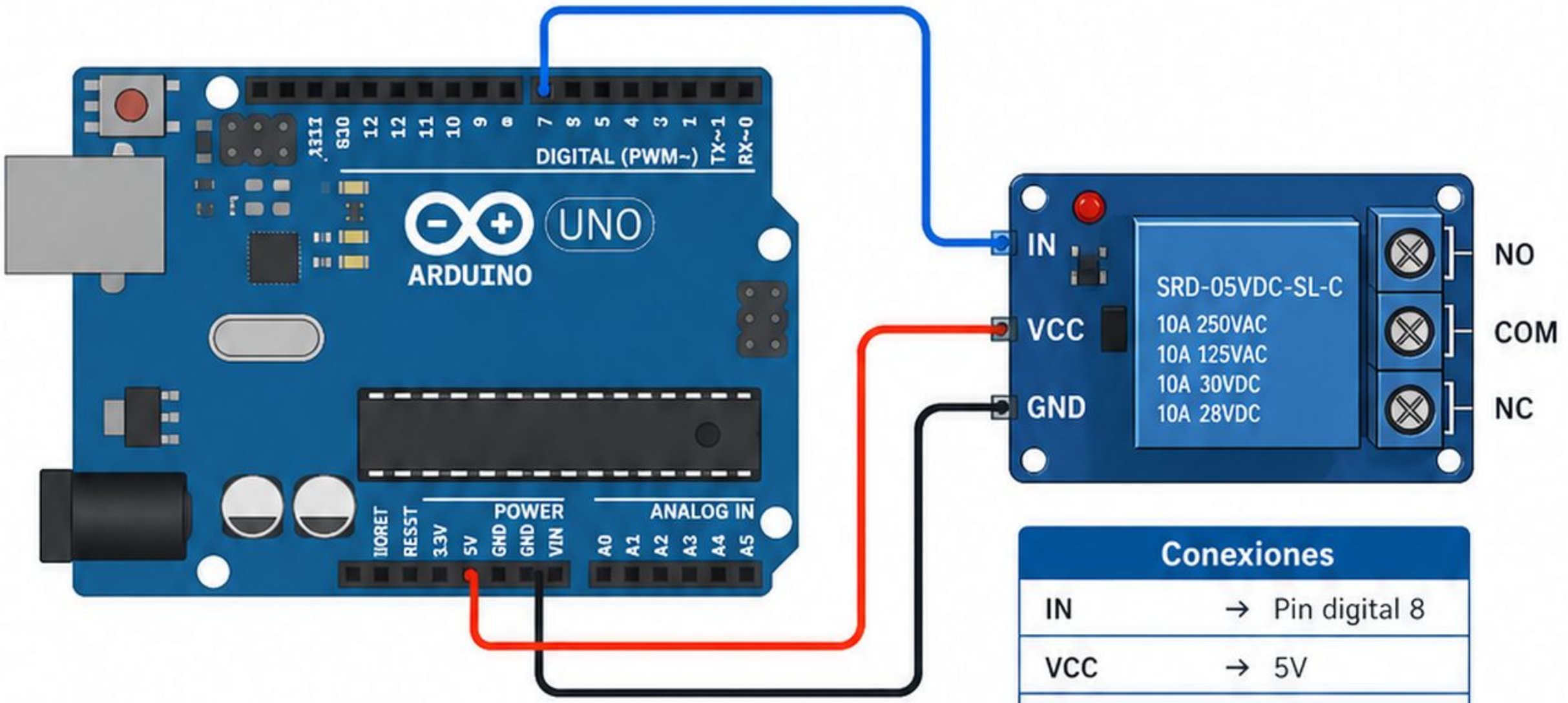
1. Abre el Monitor Serial en el IDE de Arduino. Configura la velocidad en 9600 baudios.
2. Escribe un carácter o texto y envíalo. (Asegúrate de tener “Nueva línea” activado).
3. Arduino recibirá el dato y lo mostrará en el Monitor Serial con el prefijo “Recibido: ”.

TIP Esta plantilla es útil para depuración, ingreso de comandos y comunicación básica entre Arduino y la PC.

Plantilla:
Controlar un Relé

Esta plantilla permite encender y apagar un relé conectado a un pin digital del Arduino.

DIAGRAMA DE CONEXIÓN



Conexiones		
IN	→	Pin digital 8
VCC	→	5V
GND	→	GND
NO / COM / NC	→	A la carga

NO: Normalmente Abierto
NC: Normalmente Cerrado

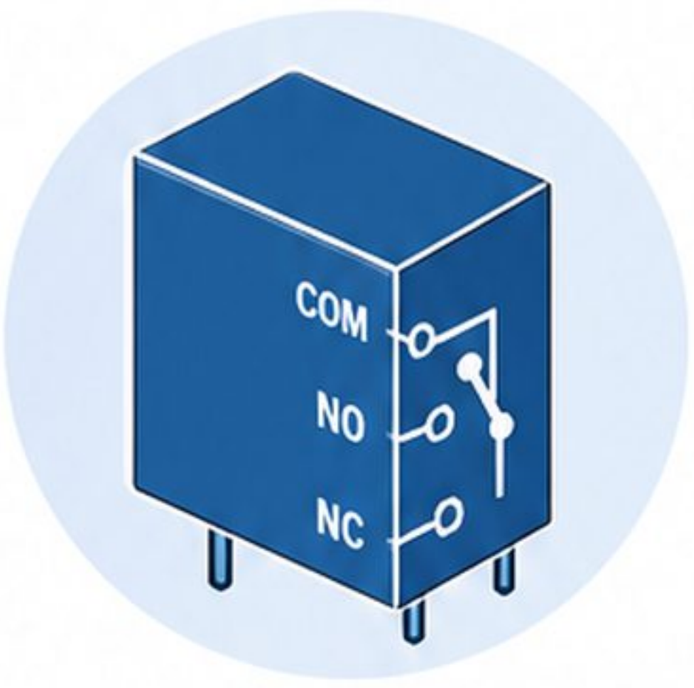
CÓDIGO DE EJEMPLO

```
int pin = 8;

void setup() {
  pinMode(pin, OUTPUT);
}

void loop() {
  digitalWrite(pin, HIGH);
  delay(5000);
  digitalWrite(pin, LOW);
  delay(5000);
}
```

¿QUÉ ES UN RELÉ?



Un relé es un interruptor controlado eléctricamente que permite encender o apagar cargas de mayor voltaje o corriente de forma segura.

ARDUINO

Plantilla: Conectar un Display LCD





En esta plantilla aprenderás a conectar un display LCD 16x2 con módulo I2C y mostrar un mensaje usando la librería LiquidCrystal.

DIAGRAMA DE CONEXIÓN

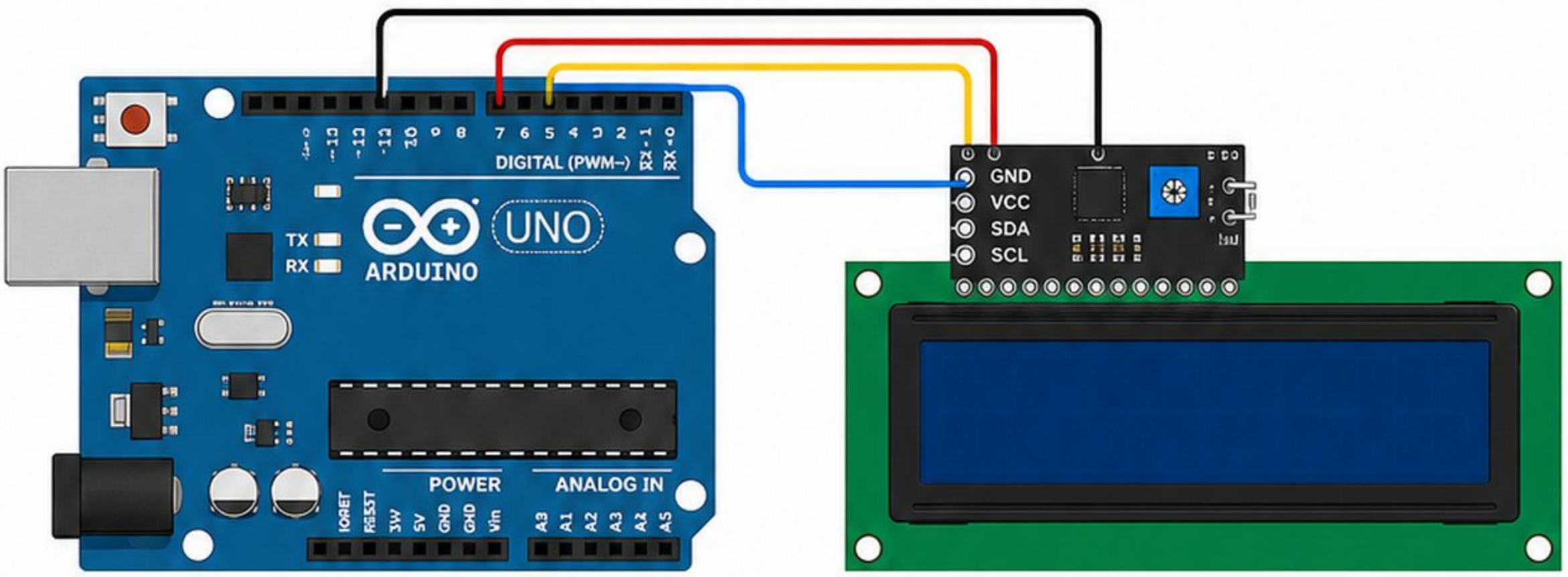


TABLA DE CONEXIONES

LCD I2C	ARDUINO UNO	DESCRIPCIÓN
GND	GND	Tierra (0V)
VCC	5V	Alimentación (5V)
SDA	A4	Datos I2C (SDA)
SCL	A5	Reloj I2C (SCL)

NOTA

Este ejemplo usa un módulo LCD 16x2 con interfaz I2C (PCF8574). Solo necesitas 4 conexiones.

CÓDIGO DE EJEMPLO

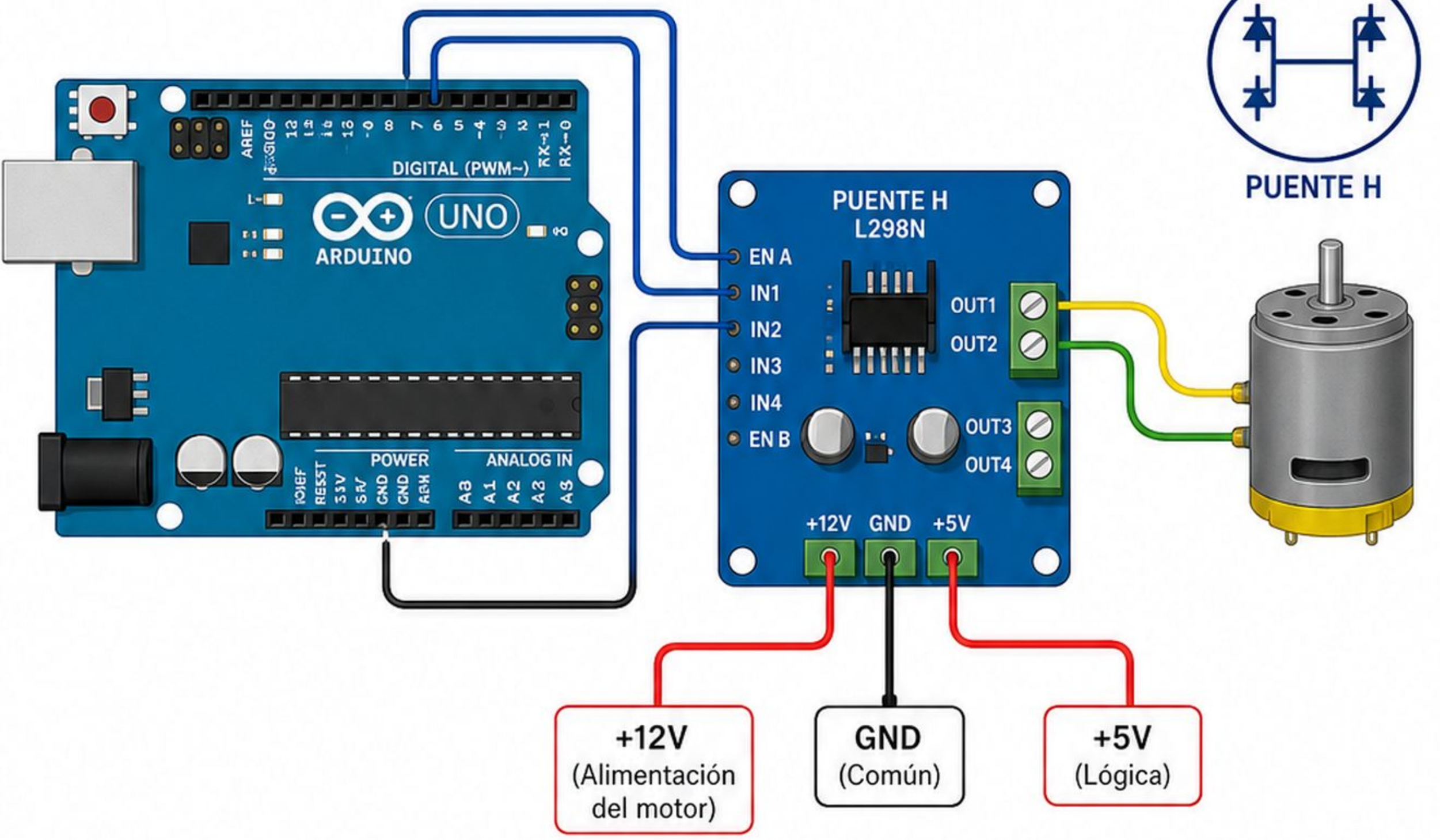
```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);
  lcd.print("Hola Mundo!");
}

void loop() {
}
```

Plantilla: Controlar un Motor DC con Puente H

DIAGRAMA DE CONEXIÓN



CÓDIGO DE EJEMPLO

```
int enA = 9;
int in1 = 8;
int in2 = 7;

void setup() {
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
}

void loop() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  analogWrite(enA, 255);
  delay(2000);
  analogWrite(enA, 0);
  delay(2000);
}
```

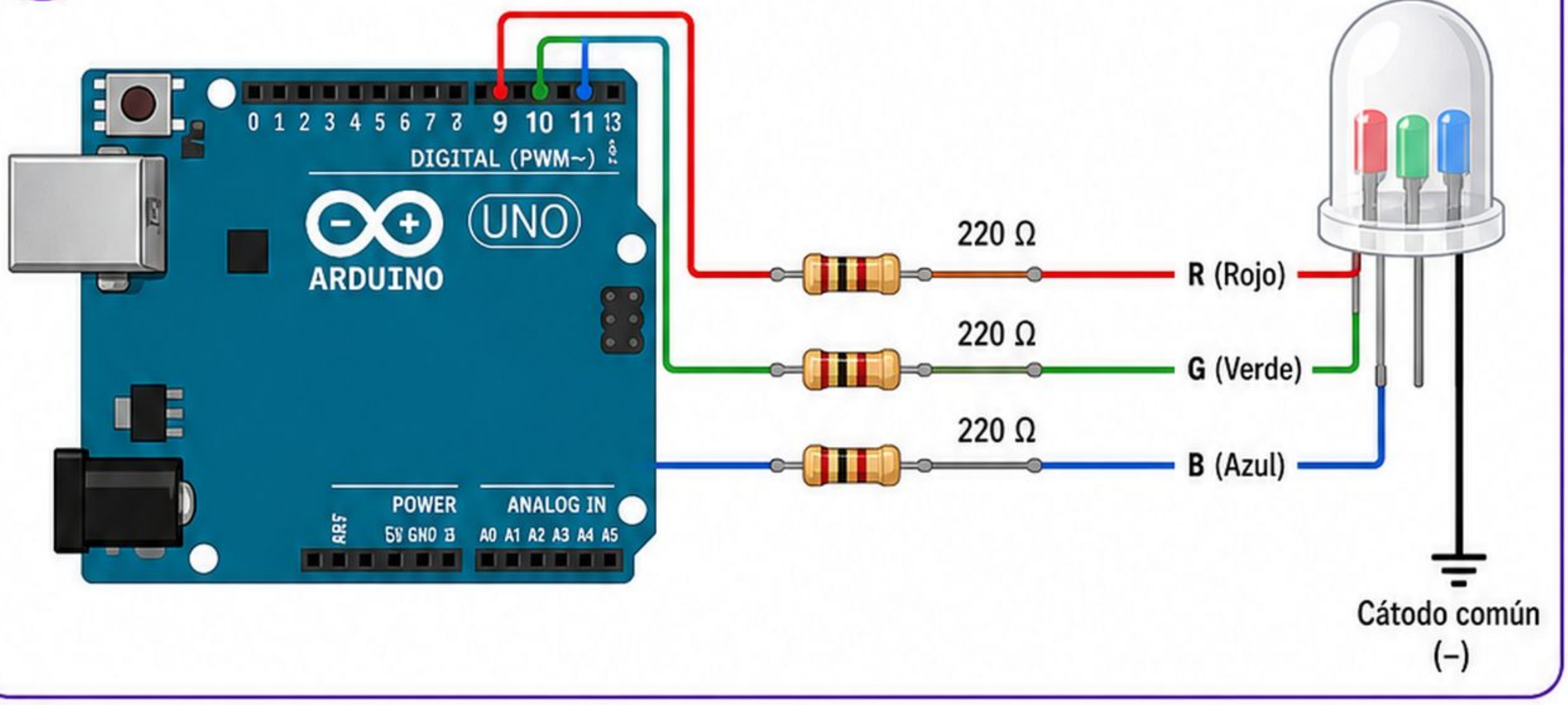
Explicación:

- Activa el motor en un sentido a máxima velocidad durante 2 segundos.
- Detiene el motor durante 2 segundos.

Puedes invertir el sentido cambiando HIGH/LOW entre in1 e in2.

Plantilla: LED RGB con PWM

DIAGRAMA DE CONEXIÓN



CÓDIGO

```ino
int redPin = 9;
int greenPin = 10;
int bluePin = 11;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

void loop() {
  setColor(255, 0, 0);     // Rojo
  delay(1000);
  setColor(0, 255, 0);     // Verde
  delay(1000);
  setColor(0, 0, 255);     // Azul
  delay(1000);
}

void setColor(int red, int green, int blue) {
  analogWrite(redPin, red);
  analogWrite(greenPin, green);
  analogWrite(bluePin, blue);
}
```

LED RGB



PALETA DE COLORES



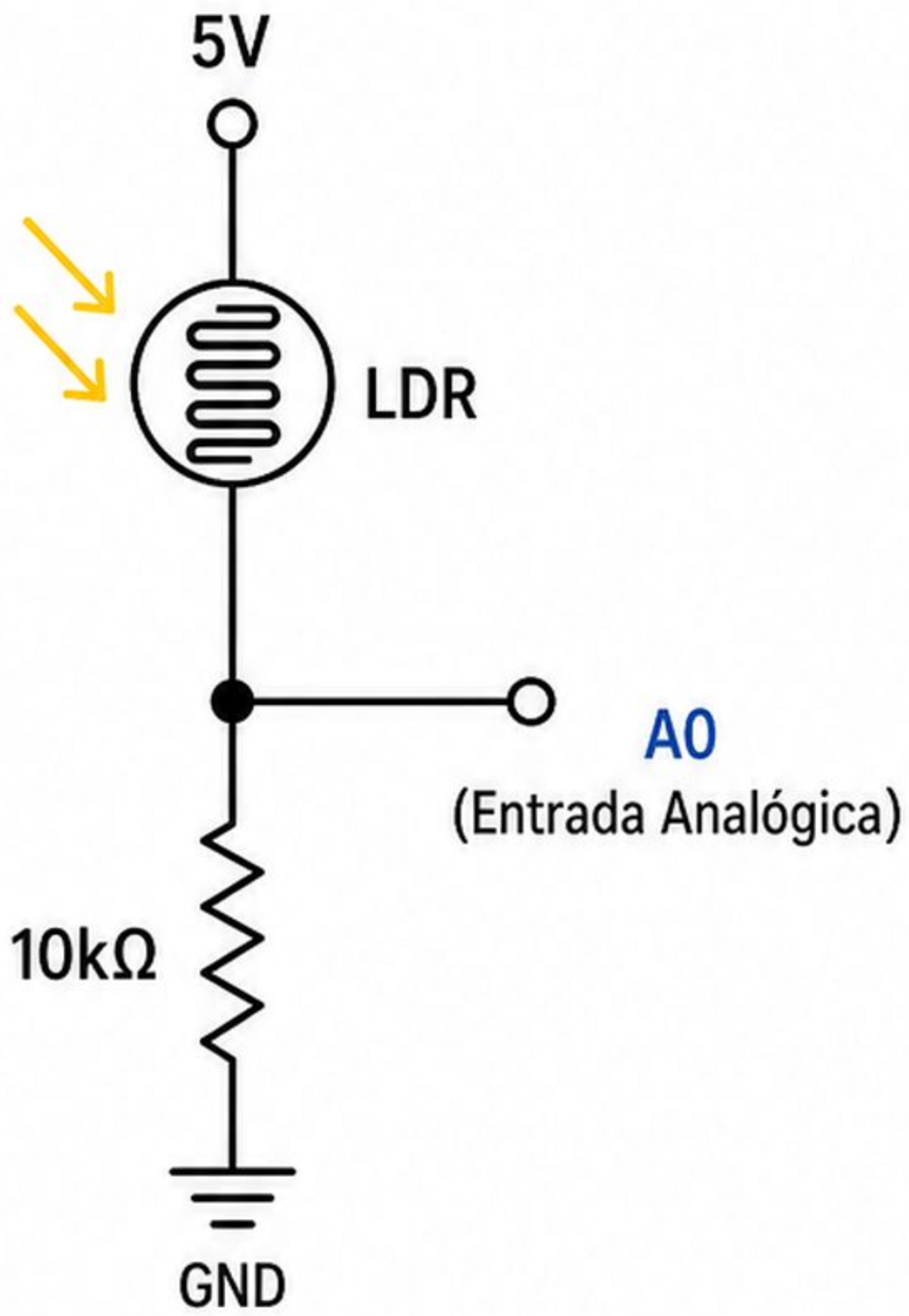


Usa valores entre 0 y 255 para mezclar colores.

Plantilla:
Sensor de Luz LDR

DIAGRAMA DE CONEXIÓN

Divisor de tensión con LDR y resistencia de 10kΩ a A0.



CÓDIGO (ARDUINO)

```ino
int ldrPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(ldrPin);
  Serial.print("Luz: ");
  Serial.println(valor);

  if (valor < 500) {
    Serial.println("Oscuro");
  } else {
    Serial.println("Claro");
  }
  delay(500);
}
```



CLARO
Más luz
valor alto
(>= 500)



OSCURO
Menos luz
valor bajo
(< 500)

Ajusta el umbral según tu entorno.

Plantilla:
Pantalla LCD con Comunicación I2C

DIAGRAMA DE CONEXIÓN



CONEXIONES I2C

SDA	→	A4
SCL	→	A5
VCC	→	5V
GND	→	GND

</> CÓDIGO DE EJEMPLO

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

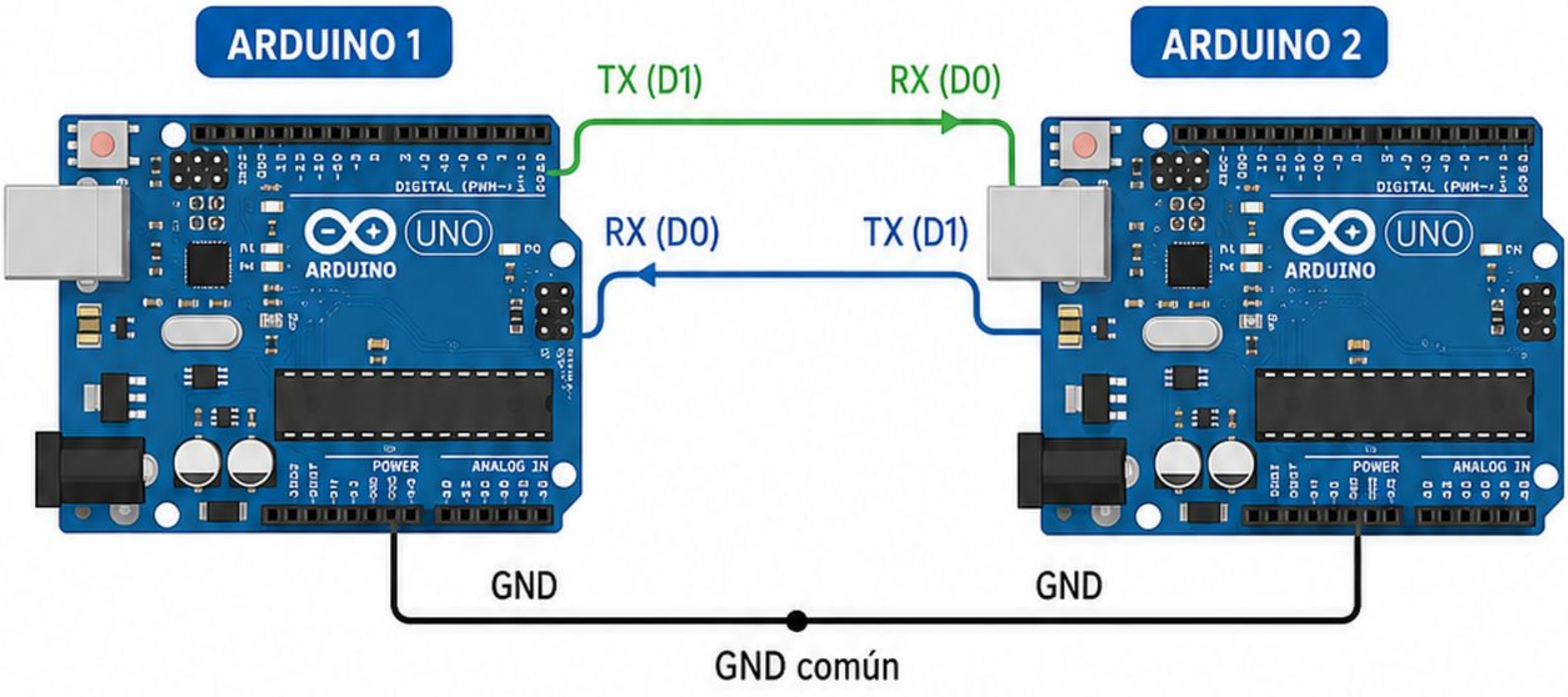
void setup() {
  lcd.init();
  lcd.backlight();
  lcd.print("Hola Mundo!");
}

void loop() {
  lcd.setCursor(0, 1);
  lcd.print(millis() / 1000);
  delay(1000);
}
```

Ajusta la dirección I2C si es diferente en tu módulo.

Plantilla: Comunicación Serial entre Dos Arduinos

Conecta dos Arduinos usando los pines RX y TX con GND común para comunicación serial.



Código (Ejemplo receptor)

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    char dato = Serial.read();
    Serial.print("Recibido: ");
    Serial.println(dato);
  }
}
```

Usa el monitor serial para depurar o enviar datos.

Consejos y Trucos
Finales



1. Usa constantes para pines



2. Organiza tu código en funciones



3. Comenta tus bloques



4. Prueba con protoboard antes de soldar



5. Guarda versiones de tus proyectos



¡Sigue practicando y crea tus propias plantillas!